

Access Modifiers in Java

There are two types of modifiers in Java: **access modifiers** and **non-access modifiers**.

The access modifiers in Java specifies the accessibility or scope of a field, method, constructor, or class. We can change the access level of fields, constructors, methods, and class by applying the access modifier on it.

There are four types of Java access modifiers:

- ❑ **Private:** The access level of a private modifier is only within the class. It cannot be accessed from outside the class.
- ❑ **Default:** The access level of a default modifier is only within the package. It cannot be accessed from outside the package. If you do not specify any access level, it will be the default.
- ❑ **Protected:** The access level of a protected modifier is within the package and outside the package through child class. If you do not make the child class, it cannot be accessed from outside the package.
- ❑ **Public:** The access level of a public modifier is everywhere. It can be accessed from within the class, outside the class, within the package and outside the package.

Access Modifier	within class	within package	outside package by subclass only	outside package
Private	Y	N	N	N
Default	Y	Y	N	N
Protected	Y	Y	Y	N
Public	Y	Y	Y	Y



- **// Calculator.java**
- **public class Calculator {**
- **// Public method to add two numbers**
- **public int add(int a, int b) {**
- **return a + b;**
- **}**
- **}**
- **// TestCalculator.java**
- **public class TestCalculator {**
- **public static void main(String[] args) {**
- **Calculator calc = new Calculator();**
- **int result = calc.add(5, 3); // Accessing the**
public method
- **System.out.println("The sum is: " +**
result);
- **}**
- **}**

PRIVATE

```
class A{  
    private int data=40;  
    private void msg()  
    {  
        System.out.println("Hello java");  
    }  
}  
  
public class Simple{  
    public static void main(String args[]){  
        A obj=new A();  
        System.out.println(obj.data);//Compile Time Error  
        obj.msg();//Compile Time Error  
    }  
}
```

Simple example of private access modifier

In this example, we have created two classes A and Simple. A class contains private data member and private method. We are accessing these private members from outside the class, so there is a compile-time error.

DEFAULT

//save by A.java

```
package pack;
```

```
class A{
```

```
    void msg(){System.out.println("Hello");
```

```
}
```

//save by B.java

```
package mypack;
```

```
import pack.*;
```

```
class B{
```

```
    public static void main(String args[]){
```

```
        A obj = new A();//Compile Time Error
```

```
        obj.msg();//Compile Time Error
```

```
    } }
```

If you don't use any modifier, it is treated as **default** by default. The default modifier is accessible only within package. It cannot be accessed from outside the package. It provides more accessibility than private. But, it is more restrictive than protected, and public.

Example of default access modifier

In this example, we have created two packages pack and mypack. We are accessing the A class from outside its package, since A class is not public, so it cannot be accessed from outside the package.

3) Protected

The protected access modifier is accessible within package and outside the package but through inheritance only.

The protected access modifier can be applied on the data member, method and constructor. It can't be applied on the class.

It provides more accessibility than the default modifier.

Example of protected access modifier

Let us assume, we have created the two packages pack and mypack. The A class of pack package is public, so can be accessed from outside the package. But msg method of this package is declared as protected, so it can be accessed from outside the class only through inheritance.

//save by A.java

```
package pack;  
public class A{  
protected void msg()  
{  
System.out.println("Hello");  
}
```

Output:Hello

//save by B.java

```
package mypack;  
import pack.*;  
class B extends A{  
  public static void main(String args[]){  
    B obj = new B();  
    obj.msg();  
  } }
```

Example of public access modifier

1. **//save by A.java**

2.

3. **package** pack;

4. **public class** A{

5. **public void** msg(){System.out.println("Hello");}

6. }

7. **//save by B.java**

8.

9. **package** mypack;

10. **import** pack.*;

11.

12. **class** B{

13. **public static void** main(String args[]){

14. A obj = **new** A();

15. obj.msg();

16. }

17. }

4) Public

The **public** access modifier is accessible everywhere. It has the widest scope among all other modifiers.

Methods

in

Java

Method in Java

In general, a method is a way to perform some task. Similarly, the method in Java is a collection of instructions that performs a specific task. It provides the reusability of code. We can also easily modify code using methods.

What is a method in Java?

A method is a block of code or collection of statements or a set of code grouped together to perform a certain task or operation. It is used to achieve the reusability of code. **We write a method once and use it many times.** We do not require to write code again and again. It also provides the easy modification and readability of code, just by adding or removing a chunk of code. The method is executed only when we call or invoke it.

The most important method in Java is the `main()` method.

Definition and Syntax

A method in Java can be defined as a collection of statements grouped together to perform an operation. Each method must be part of a class.

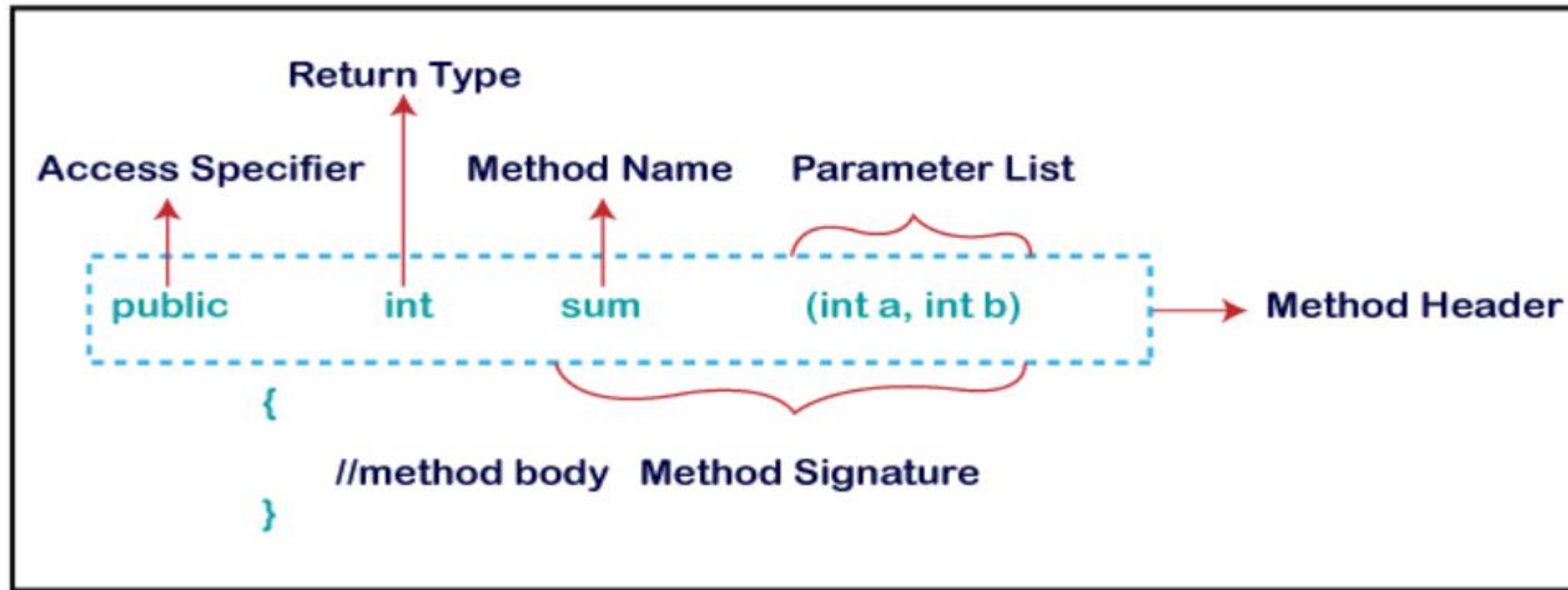
A method's basic syntax is:

```
accessModifier returnType methodName(parameterList) {  
    // method body  
}
```

Method Declaration

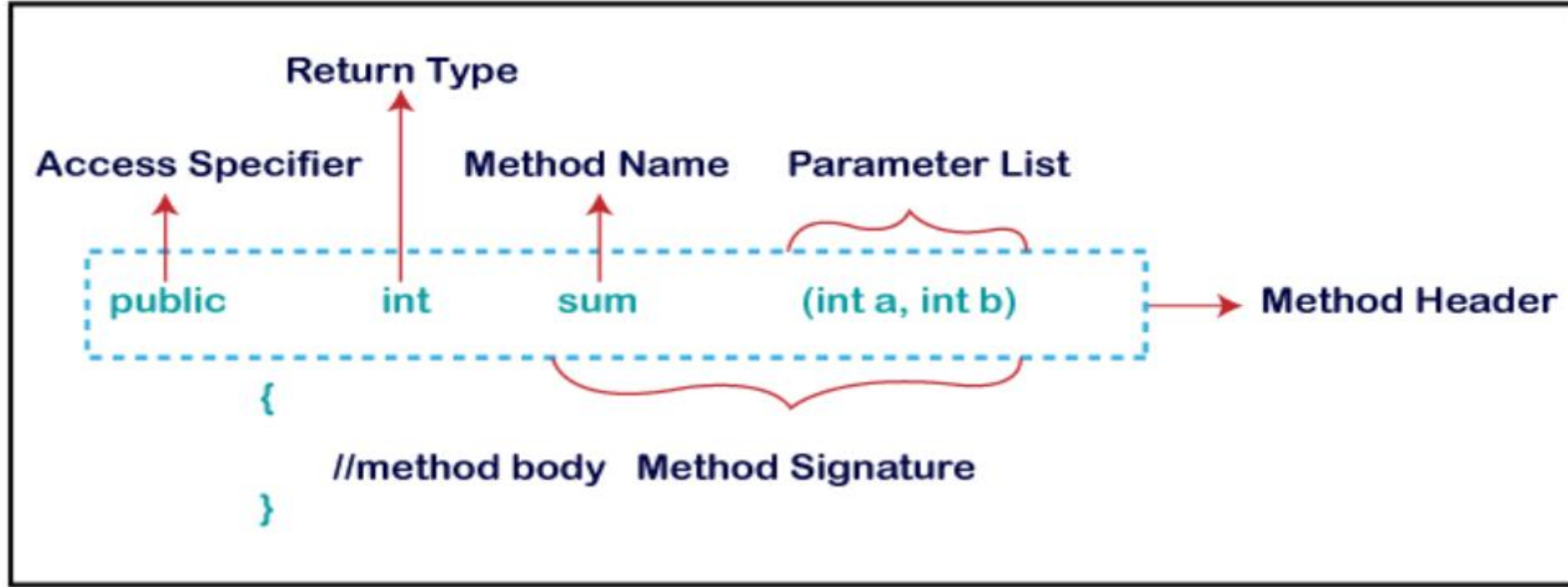
The method declaration provides information about method attributes, such as visibility, return-type, name, and arguments. It has six components that are known as method header,

Method Declaration

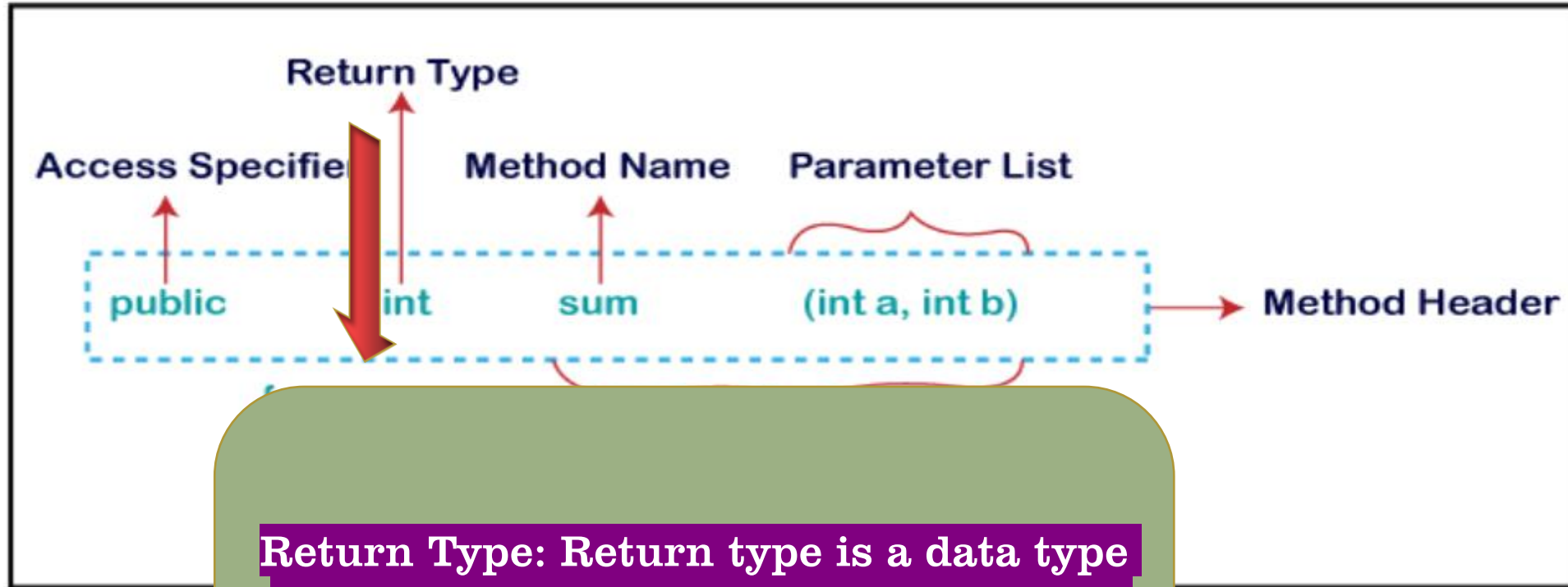


Method Signature: Every method has a method signature. It is a part of the method declaration. It includes the method name and parameter list.

Method Declaration

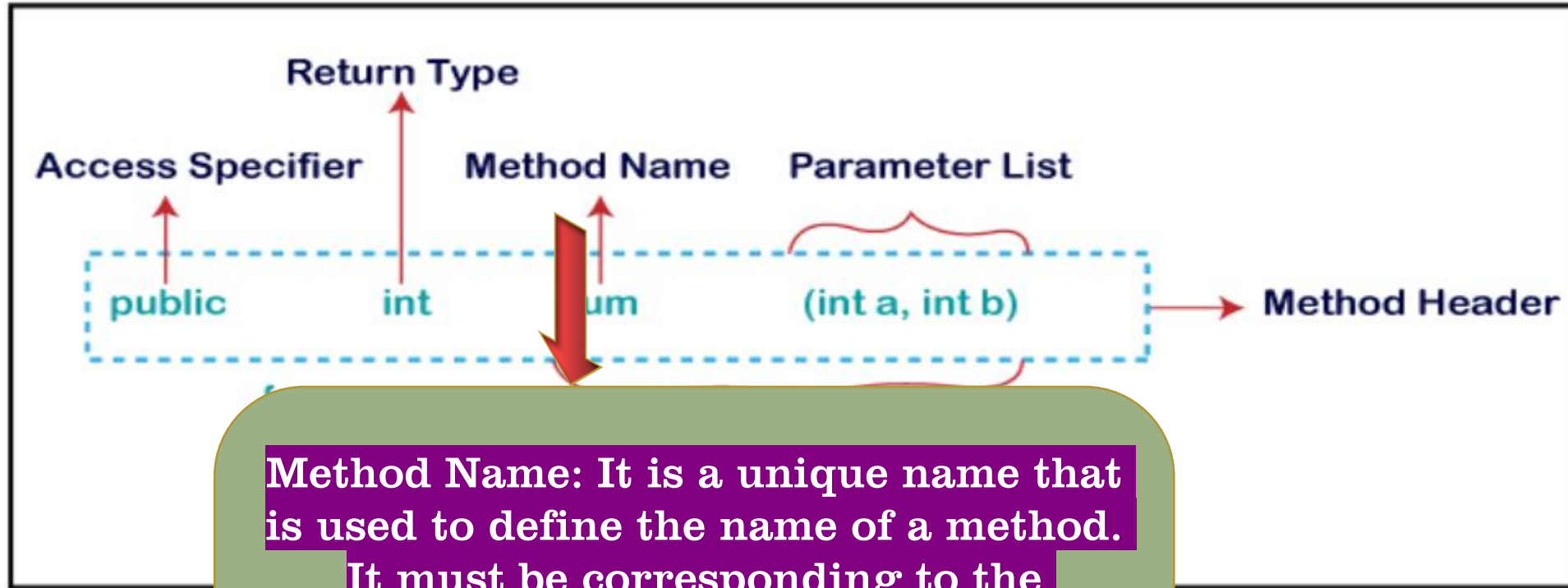


Method Declaration



Return Type: Return type is a data type that the method returns. It may have a primitive data type, object, collection, void, etc. If the method does not return anything, we use void keyword

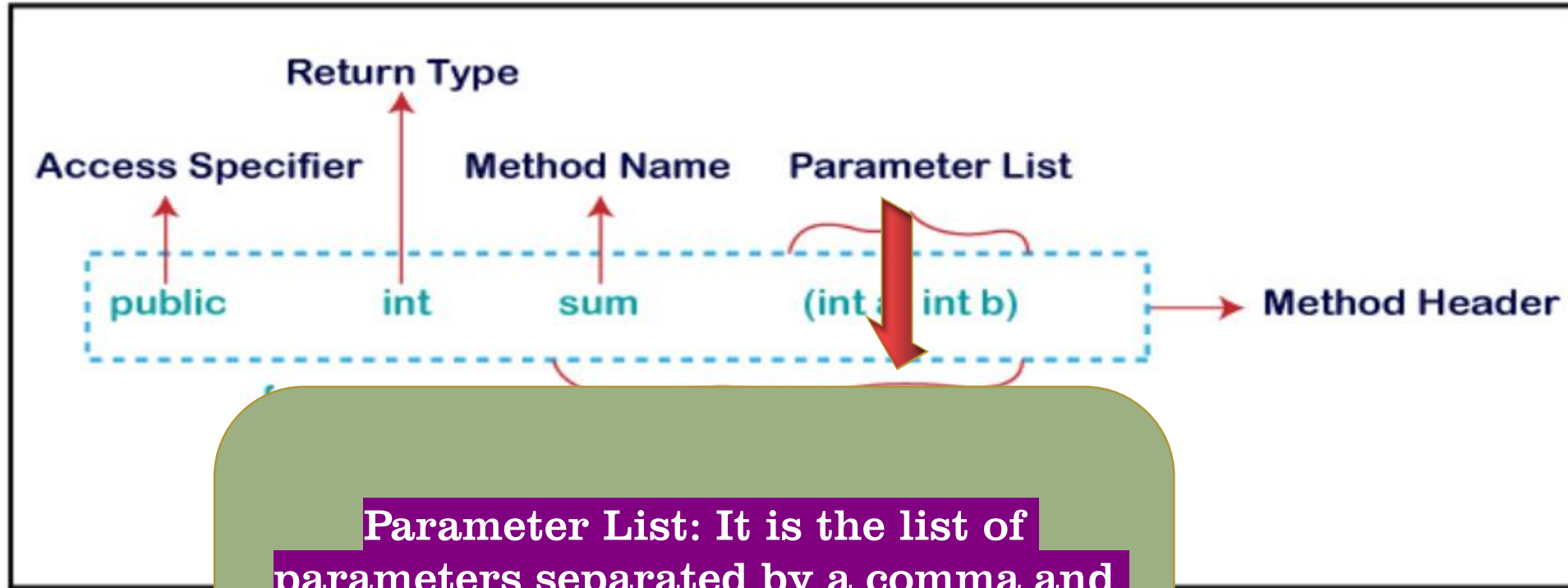
Method Declaration



Method Name: It is a unique name that is used to define the name of a method.

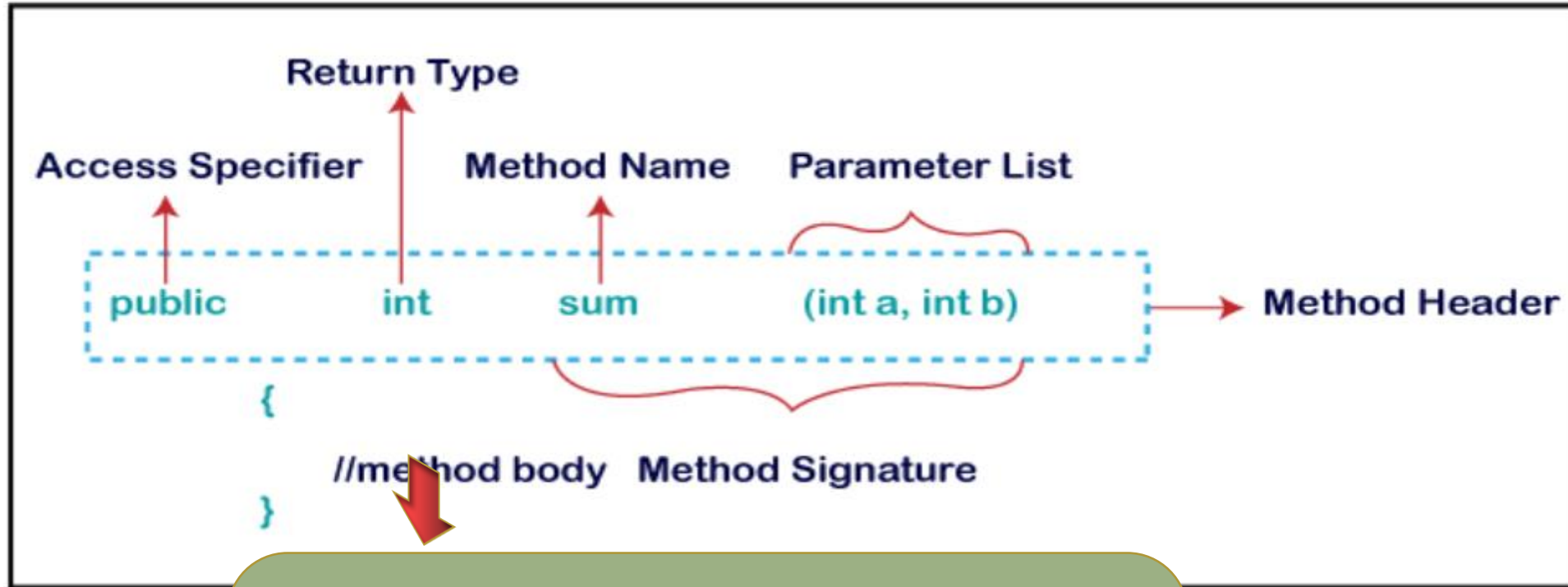
It must be corresponding to the functionality of the method. Suppose, if we are creating a method for subtraction of two numbers, the method name must be `subtraction()`. A method is invoked by its name.

Method Declaration



Parameter List: It is the list of parameters separated by a comma and enclosed in the pair of parentheses. It contains the data type and variable name. If the method has no parameter, left the parentheses blank.

Method Declaration



Method Body: It is a part of the method declaration. It contains all the actions to be performed. It is enclosed within the pair of curly braces.

Types of Method

There are two types of methods in Java:

❖ Predefined Method

❖ User-defined Method

- **Predefined
Method**



- **Predefined Method**

- In Java, predefined methods are the method that is already defined in the Java class libraries is known as predefined methods. It is also known as the standard library method or built-in method. We can directly use these methods just by calling them in the program at any point. Some pre-defined methods are `length()`, `equals()`, `compareTo()`, `sqrt()`, etc. When we call any of the predefined methods in our program, a series of codes related to the corresponding method runs in the background that is already stored in the library.



Demo.java

```
public class Demo
{
public static void main(String[] args)
{
// using the max() method of Math class
System.out.print("The maximum number is: " + Math.max(9,7));
} }
```

Demo.java

```
public class Demo
{
    public static void main(String[] args)
    {
        // using the max() method of Math class
        System.out.print("The maximum number is: " + Math.max(9,7)),
    } }
```

Demo.java

```
public class Demo
{
    public static void main(String[] args)
    {
        // using the max() method of Math class
        System.out.print("The maximum number is: " + Math.max(9,7));
    } }
```

Demo.java

```
public class Demo
{
    public static void main(String[] args)
    {
        // using the max() method of Math class
        System.out.print("The maximum number is: " + Math.max(9,7));
    } }
```

The maximum number is: 9

In the above example, we have used three predefined methods `main()`, `print()`, and `max()`. We have used these methods directly without declaration because they are predefined. The `print()` method is a method of `PrintStream` class that prints the result on the console. The `max()` method is a method of the `Math` class that returns the greater of two numbers.



1. Object Class



equals(Object obj): Determines whether two objects are equal.



hashCode(): Returns a hash code value for the object.



toString(): Returns a string representation of the object.



getClass(): Gets the runtime class of an object.



clone(): Creates and returns a copy of this object.

String Class

`charAt(int index)`: Returns the character at the specified index.

`length()`: Returns the length of the string.

`substring(int beginIndex, int endIndex)`: Returns a substring of the string.

`indexOf(String str)`: Returns the index within the string of the first occurrence of the specified substring.

`toLowerCase()`: Converts all characters in the string to lower case.

`toUpperCase()`: Converts all characters in the string to upper case.

`split(String regex)`: Splits the string around matches of the given regular expression

- **Math Class (Static Methods)**

- **sqrt(double a):** Returns the square root of a value.

- **pow(double a, double b):** Returns the value of the first parameter raised to the power of the second parameter.

- **abs(int a):** Returns the absolute value of an integer.

- **max(int a, int b):** Returns the greater of two int values.

- **min(int a, int b):** Returns the smaller of two int values.

- **round(double a):** Rounds a double value to the nearest whole number.

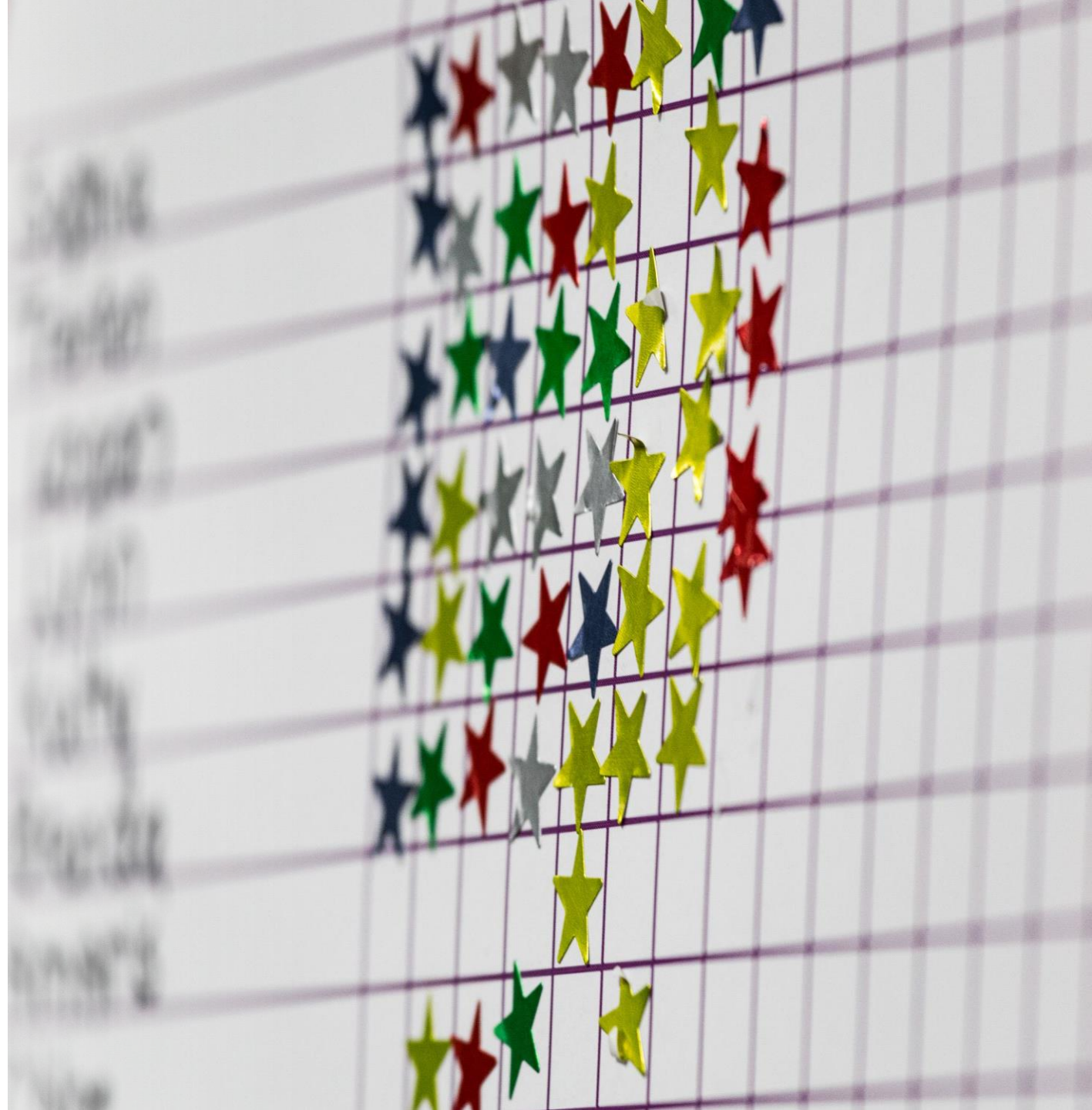


- **Arrays Class (Static Methods)**

- **sort(int[] a):** Sorts the specified array of ints into ascending numerical order.
- **binarySearch(int[] a, int key):** Searches the specified array for the specified value using the binary search algorithm.
- **copyOf(T[] original, int newLength):** Copies the specified array, truncating or padding with nulls (if necessary) so the copy has the specified length.
- **equals(int[] a, int[] a2):** Returns true if the two specified arrays of ints are equal to one another.
- **fill(int[] a, int val):** Assigns the specified int value to each element of the specified array of ints.

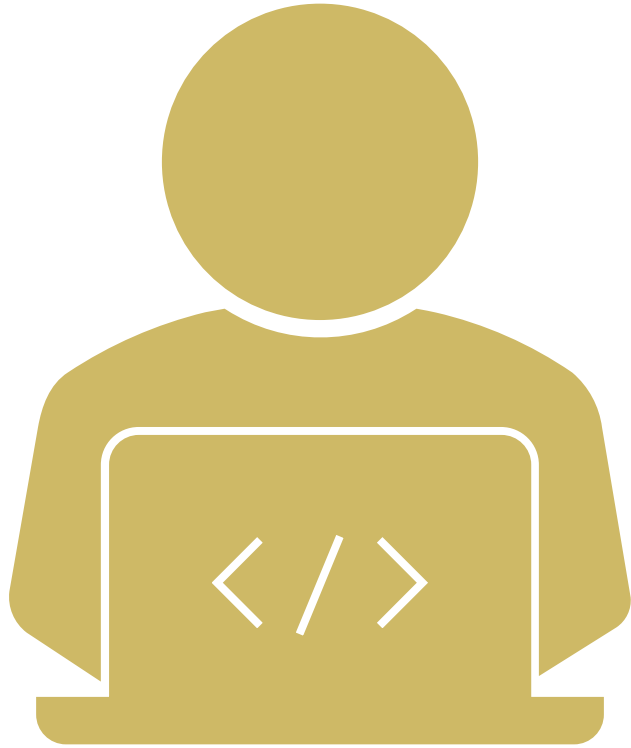
• Collections Framework

- **add**(E e), **remove**(Object o), **size**(), **isEmpty**(): Basic operations provided by Collection interface implementations like ArrayList, HashSet, etc.
- **sort**(List<T> list): Provided by Collections class to sort the specified list into ascending order.
- **binarySearch**(List<? extends Comparable<? super T>> list, T key): Searches the specified list for the specified object using the binary search algorithm.





User - defined Methods



- **User-defined Method**

- The method written by the user or programmer is known as a user-defined method. These methods are modified according to the requirement.

- **Defining a User-defined Method:**

- A user-defined method is declared within a class, and its structure includes an access modifier, return type, method name, parameter list (if any), and a body enclosed in braces.

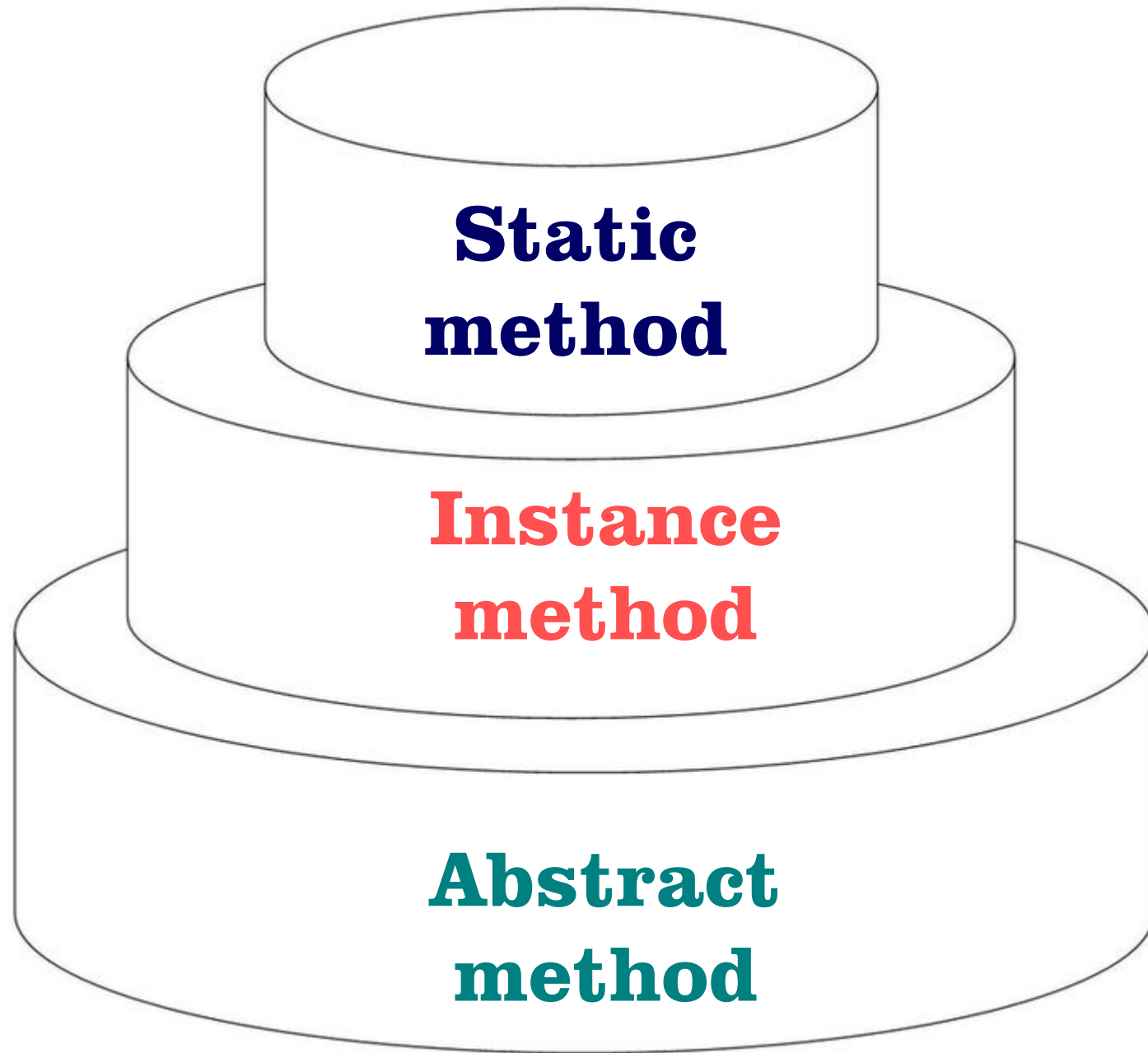
- ```
public class MyClass {
```
- ```
    // An example of a user-defined method
```
- ```
 public static int addNumbers(int a, int b) {
```
- ```
        int sum = a + b;
```
- ```
 return sum;
```
- ```
    }
```
- ```
}
```

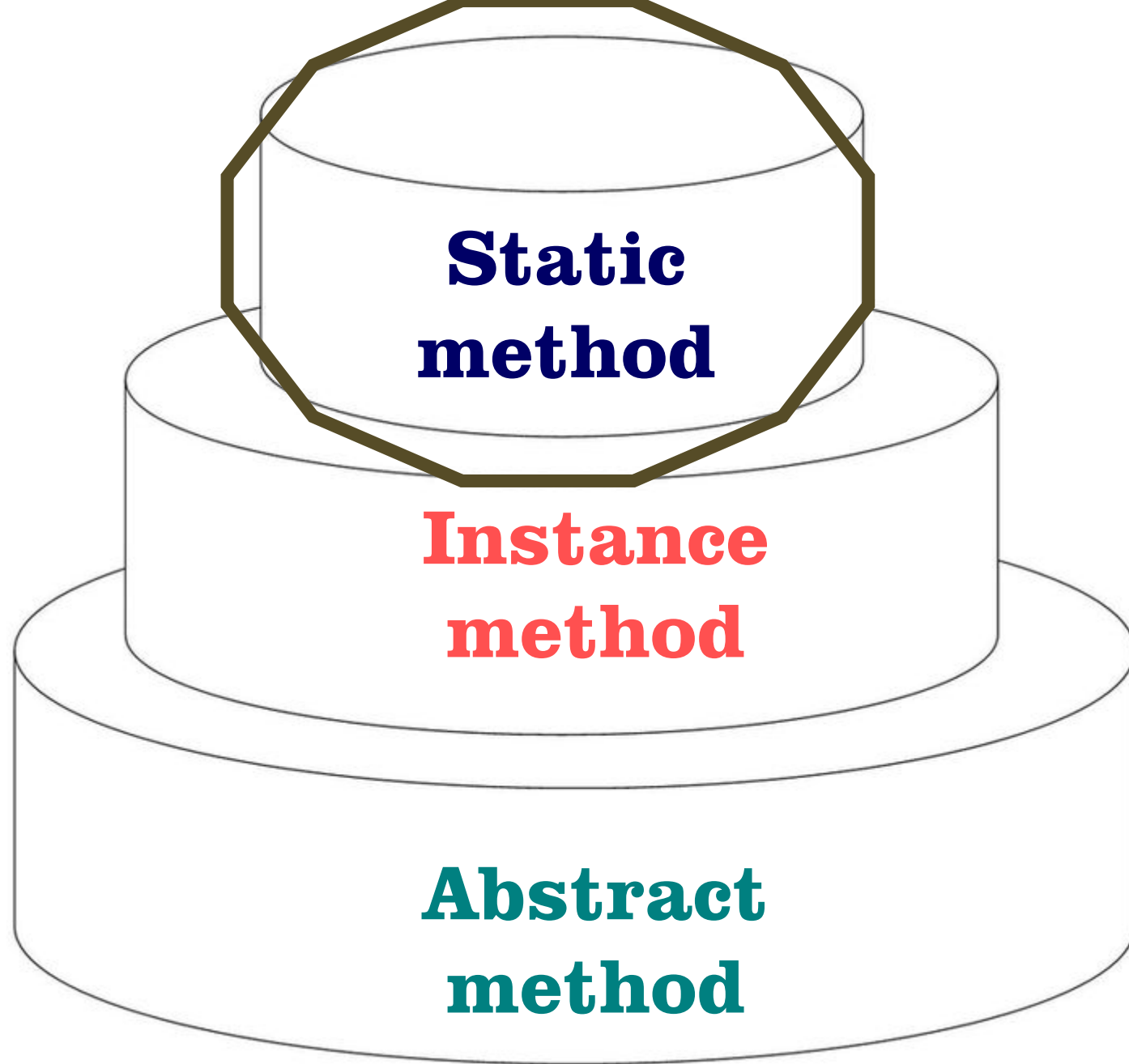
## Calling a User-defined Method:

Once defined, a user-defined method can be called by specifying its name and passing the required arguments (if the method accepts any parameters).

```
public class Test {
 public static void main(String[] args) {
 int result = MyClass.addNumbers(5, 10); // calling
the method
 System.out.println("The sum is: " + result);
 }
}
```







# Static Methods

Static methods belong to the class rather than a specific instance of the class. This means that they can be called without creating an object of the class. Static methods are used for operations that don't require any data from an instance of the class (object).

Syntax:

```
public static returnType methodName(parameters)
{
 // body
}
```

**Example: Checking if a number is even**

```
public class Check {
 public static boolean isEven(int number) {
 return number % 2 == 0;
 }

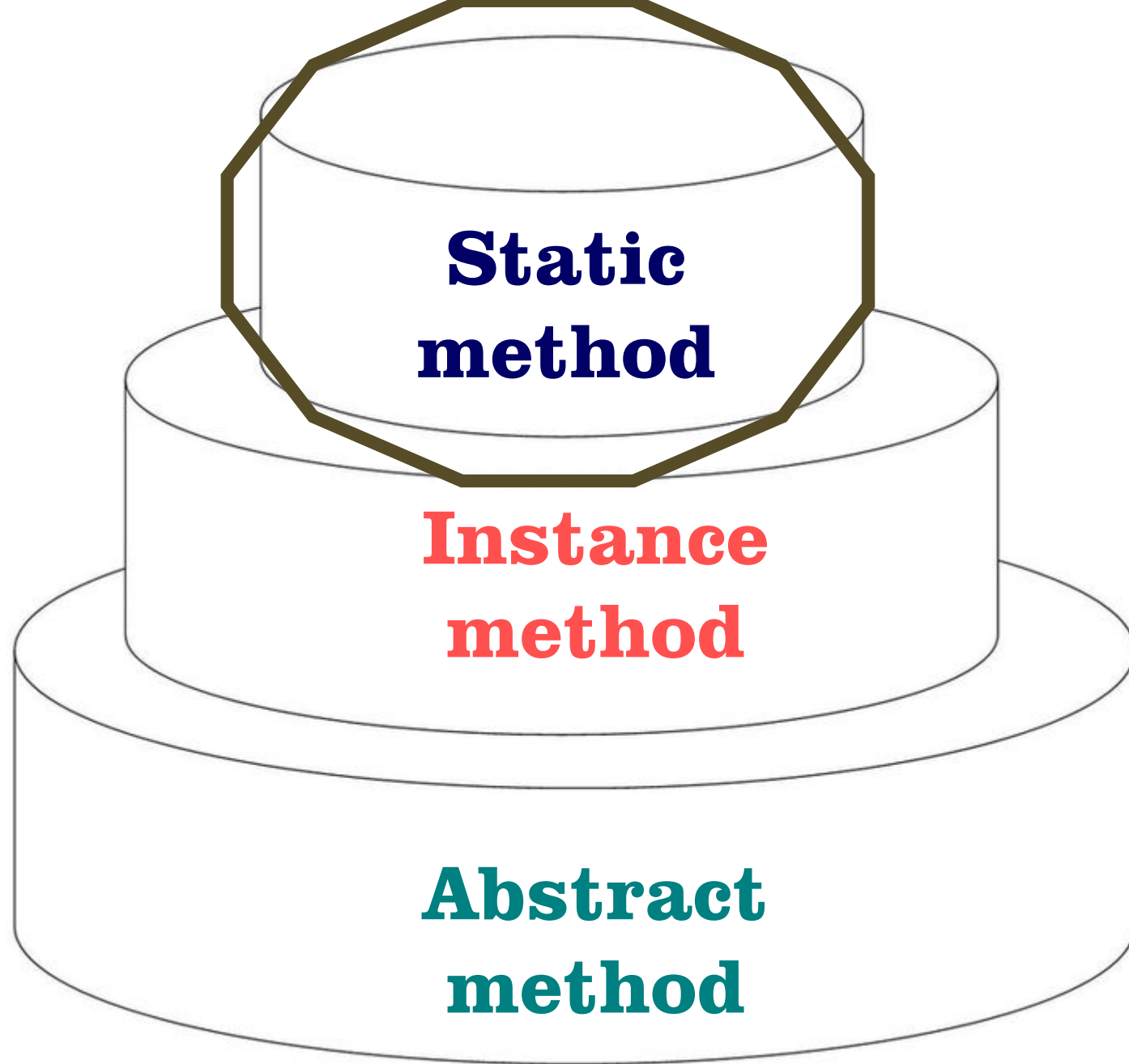
 public static void main(String[] args) {
 // This line creates a boolean variable named result. It calls the isEven method of the Check class
 // with the value 4 as its argument. The result of isEven(4) will be stored in the result variable.
 // Since 4 is an even number, isEven will return true, and true will be stored in result.
 boolean result = Check.isEven(4);
 System.out.println(result);
 }
}
```

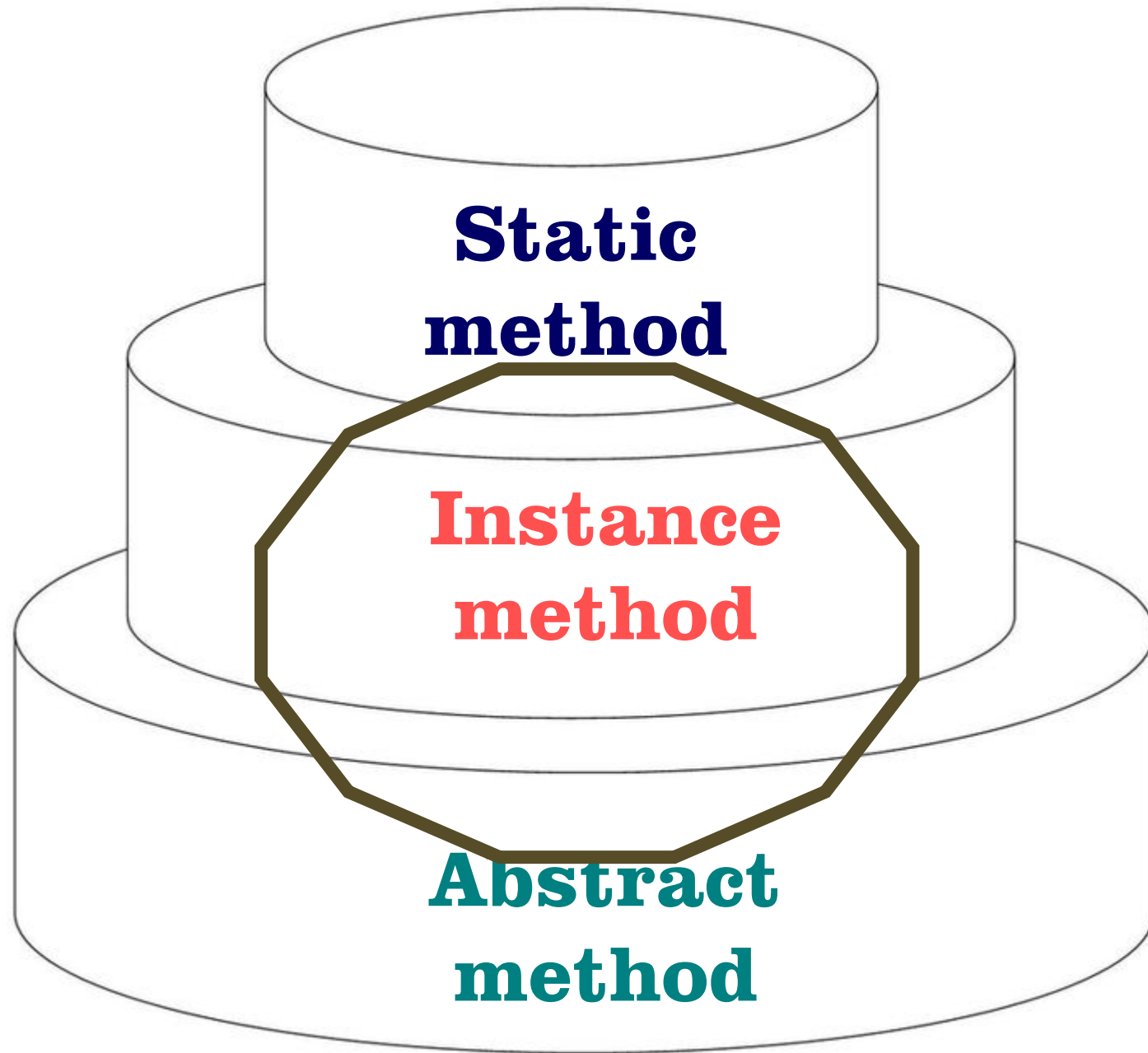
**// Output: true**

## Example: Converting temperature from Celsius to Fahrenheit

```
public class Converter {
 public static double celsiusToFahrenheit(double celsius) {
 return (celsius * 9/5) + 32;
 }

 public static void main(String[] args) {
 double fahrenheit = Converter.celsiusToFahrenheit(0);
 System.out.println(fahrenheit); // Output: 32.0
 }
}
```





## 2. Instance Methods

Instance methods belong to an instance of a class. To use an instance method, you must first create an object of the class. These methods can access instance variables and other instance methods directly.

**Syntax:**

```
public returnType methodName(parameters)
{
 // body
}
```



## Example 1: Updating and displaying the balance of a bank account

```
public class BankAccount {
 private double balance;

 public void deposit(double amount) {
 balance += amount;
 }

 public void displayBalance() {
 System.out.println("Current balance: " + balance);
 }

 public static void main(String[] args) {
 BankAccount account = new BankAccount();
 account.deposit(100);
 account.displayBalance(); // Output: Current balance: 100.0
 }
}
```

## Example 2: Calculating the area of a circle

```
public class Circle {
 private double radius;

 public Circle(double radius) {
 this.radius = radius;
 }

 public double calculateArea() {
 return Math.PI * radius * radius;
 }

 public static void main(String[] args) {
 Circle circle = new Circle(5);
 double area = circle.calculateArea();
 System.out.println(area);
 }
}
```

### Example 3: Changing and displaying the name of a person

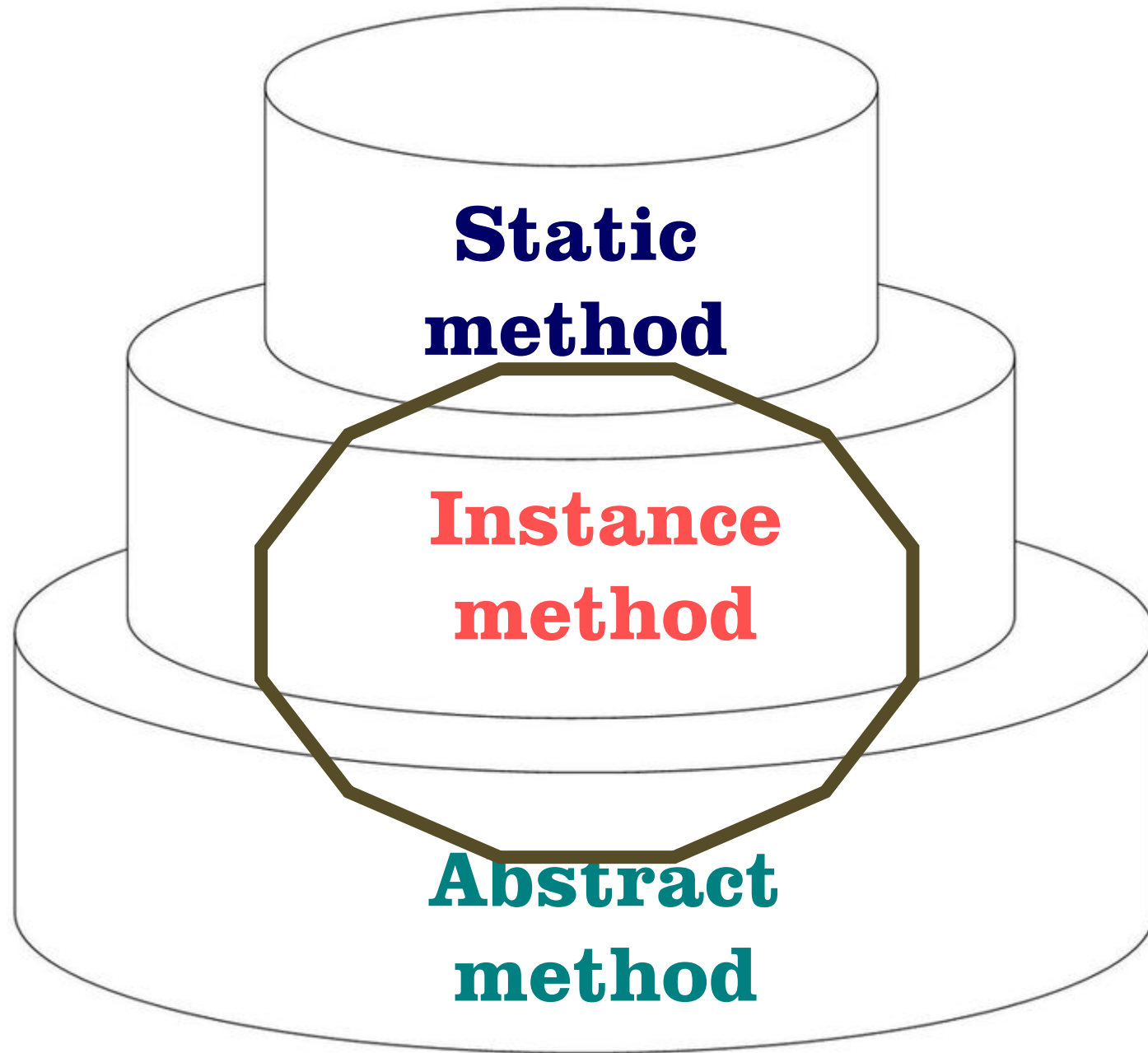
```
public class Person {
 private String name;

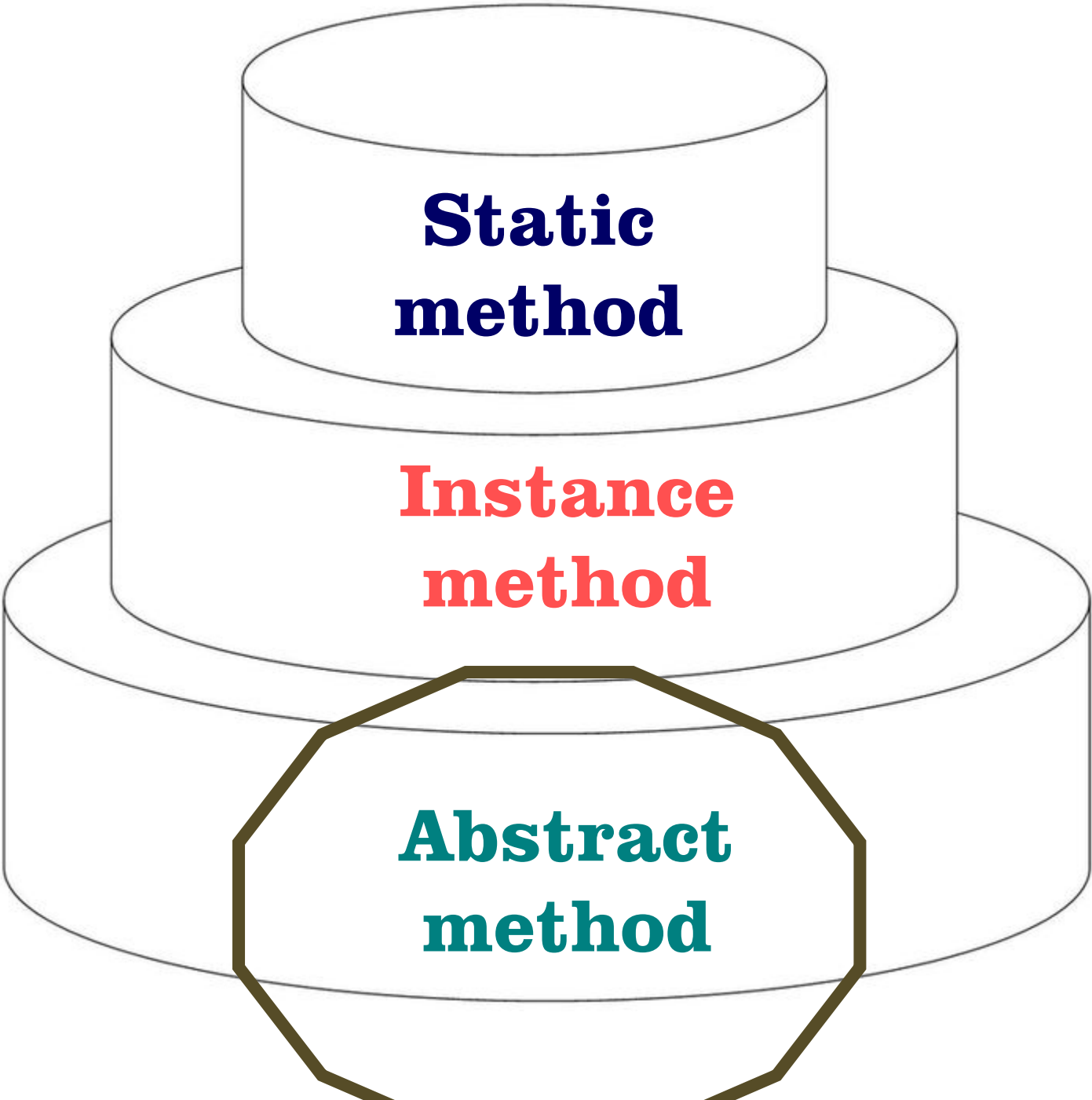
 public Person(String name) {
 this.name = name;
 }

 public void setName(String newName) {
 name = newName;
 }

 public void displayName() {
 System.out.println("Name: " + name);
 }

 public static void main(String[] args) {
 Person person = new Person("John");
 person.setName("David");
 person.displayName(); // Output: Name: David
 }
}
```





**Static  
method**

**Instance  
method**

**Abstract  
method**

### **3. Abstract Methods**

**Abstract methods are methods that are declared without an implementation. If a class includes an abstract method, then the class itself must be declared abstract. Abstract methods are used to specify that the subclasses must provide implementations for these methods.**

**Syntax:**

```
public abstract returnType methodName(parameters);
```

**Example 1: An abstract class and method for a shape**

```
public abstract class Shape {
 public abstract double calculateArea();
}
```

```
class Circle extends Shape {
 private double radius;

 public Circle(double radius) {
 this.radius = radius;
 }
 @Override
 public double calculateArea() {
 return Math.PI * radius * radius;
 }
}
```

```
class Square extends Shape {
 private double side;

 public Square(double side) {
 this.side = side;
 }
 @Override
 public double calculateArea() {
 return side * side;
 }
}
```